Home Inventory Android App

Comprehensive Development Manifest & Prompt Library

1. Executive Summary

This document serves as a comprehensive manifest and step-by-step prompt guide for developing a modern Android application dedicated to home inventory management. The app allows users to seamlessly manage household supplies by scanning product barcodes, automatically fetching product details and images, tracking stock levels, and generating automated shopping lists when supplies run low.

2. Technical Architecture & Stack

To ensure a robust, scalable, and modern Android application, the following technology stack is highly recommended. These technologies are standard in the current Android ecosystem (Modern Android Development - MAD).

Frontend UI	**Jetpack Compose** - Declarative UI framework for native Android.
Language	**Kotlin** - Fully Kotlin-based using Coroutines & Flow for asynchronous operations.
Local Storage (DB)	**Room Database** - SQLite abstraction layer for robust offline inventory storage.
Barcode Scanning	**CameraX + ML Kit Vision API** - Fast, on-device barcode and QR code scanning.
Network & API	**Retrofit2 & OkHttp** - For fetching product data from external APIs.
Image Loading	**Coil** - Image loading library backed by Kotlin Coroutines, perfect for Compose.
External Data Source	**Open Food Facts API** - Free, open database for resolving UPC/EAN barcodes to product names and images.
Background Work	**WorkManager** - For scheduling periodic stock checks and triggering low-stock notifications.

3. Core Features & User Flows

A. Barcode Scanning & Item Onboarding

The user taps a "Scan" FAB (Floating Action Button). The camera opens, instantly recognizing UPC/EAN codes. Upon successful scan, the app queries the Open Food Facts API. If found, the product name and image URL are pre-filled. The user selects the quantity and sets a "Low Stock Threshold" (e.g., alert me when less than 2 remain).

B. Inventory Dashboard

A visually appealing grid or list view showing all current items in the home. Each card displays the product image, name, current quantity, and quick "+ / -" buttons to easily update stock without opening the full item detail screen.

C. Low Stock Notifications

When an item's quantity drops below its user-defined minimum threshold, a local Android notification is triggered ("You're running low on Milk!"). Simultaneously, the item is flagged in the database.

D. Smart Shopping List

A dedicated tab that auto-populates with items that have fallen below their minimum threshold. Users can also manually add custom items. Once shopping is done, tapping "Restock" on an item updates its inventory quantity and removes it from the list.

4. The Building Process (Prompt Manifest)

Below is the structured, phase-by-phase building process. Each phase includes a **Ready-to-Use AI Prompt**. You can copy and paste these prompts sequentially into an AI coding assistant (like Cursor, GitHub Copilot, or ChatGPT) to generate the production-ready code.

Phase 1: Project Setup & Database Architecture

The foundation of the app relies on a solid local database. We need two primary entities: `InventoryItem` and `ShoppingListItem`.

```
Act as an expert Android developer. I am building a Home Inventory app using Kotlin and Jetpack Compose.

Create the Room Database setup for this app. Include the following:
1. An `InventoryItem` data class (Entity) with: id (Int, primary key auto-generate), barcode (String), name (String), quantity (Int), minQuantityThreshold (Int), imageUrl (String), category (String).
2. A `ShoppingListItem` data class (Entity) with: id, itemName, isChecked (Boolean), linkedBarcode (String, nullable).
3. The `InventoryDao` interface with methods: getAllItems (Flow), getItemByBarcode, insertItem, updateQuantity, deleteItem, and getLowStockItems (where quantity <= minQuantityThreshold).
4. The Room Database abstract class.

Please use modern Kotlin Coroutines and Flow for all query returns. Provide the Gradle dependencies required for Room.
```

Phase 2: Barcode Scanner Integration

Integrating the camera and parsing the barcode is a critical hardware feature. We use CameraX and Google's ML Kit to keep the parsing entirely on-device for speed and privacy.

```
Now, let's build the Barcode Scanner feature using CameraX and Google ML Kit Barcode Scanning.

Please provide a Jetpack Compose complete implementation for a `BarcodeScannerScreen`.
Requirements:
1. Ask for Camera permissions using Accompanist or the standard Compose permission APIs.
2. Implement an AndroidView that binds a CameraX `Preview` and an `ImageAnalysis` use case.
3. Attach an `MlKitAnalyzer` (or custom ImageProxy analyzer) configured to read standard retail barcodes (UPC-A, UPC-E, EAN-13, EAN-8).
4. When a barcode is detected, trigger a callback `onBarcodeScanned(barcode: String)` and pause the analyzer to prevent duplicate scans.
5. Provide the necessary Gradle dependencies for CameraX and ML Kit Vision.
```

Phase 3: External API Integration (Fetching Product Data)

Once we have the barcode string (e.g., "049000028904"), we need to resolve it into a human-readable name and an image so the user doesn't have to type it manually.

```
Next, I need to fetch product details using a barcode string from the Open Food Facts API.

1. Write a Retrofit API interface `OpenFoodFactsApi` with a GET request to: `api/v0/product/{barcode}.json`.
2. Create the necessary Kotlin serialization/Moshi/Gson data classes to parse the response. I only need the `product_name` and `image_url` fields from the JSON.
3. Write a Repository class `ProductRepository` that takes the scanned barcode, makes the network call using Coroutines, and handles Success/Error states.
4. Provide the Retrofit and OkHttp builder setup.
```

Phase 4: Inventory UI & State Management

This is the core dashboard where the user views their items. It requires fetching the local database flow and rendering a grid of cards.

```
Let's build the Main Inventory UI using Jetpack Compose and a ViewModel.

1. Create an `InventoryViewModel` that observes `getAllItems()` from the Room DAO as a StateFlow. It should also have functions to increment and decrement an item's quantity.
2. Create an `InventoryScreen` composable that displays a `LazyVerticalGrid` of `InventoryItemCard`s.
3. Each `InventoryItemCard` should display:
   - The product image loaded via the Coil library (using the `imageUrl`). Provide a placeholder if the URL is null.
   - The product name.
   - The current quantity.
   - Two icon buttons (+ and -) to immediately update the quantity.
4. Make the UI look modern, utilizing Material Design 3 cards, elevations, and proper padding.
```

Phase 5: Automated Shopping List & Notifications

The system needs to act intelligently. When the quantity drops below the threshold, it should alert the user and populate the shopping list.

> **Implementation Note:** While WorkManager can be used for periodic background syncs, since inventory levels are only changed actively by the user, the low-stock check can be run synchronously inside the ViewModel immediately after a quantity update.

```
Let's implement the low-stock logic and Shopping List UI.

Part 1: In the ViewModel, update the decrement function. When an item's quantity drops below its `minQuantityThreshold`, automatically create and insert a new `ShoppingListItem` into the database and trigger a local Android Notification saying "Stock Alert: Running low on [Item Name]". Provide the code for the Notification Channel and notification trigger.

Part 2: Create a `ShoppingListScreen` in Compose.
- It should observe a Flow of all `ShoppingListItem`s.
- Display them in a `LazyColumn`.
- Each row should have a Checkbox. When checked, it strikes through the text.
- Include a "Restock Checked Items" button at the bottom. When clicked, it should delete the checked items from the shopping list database and increment their corresponding inventory quantities in the `InventoryItem` database.
```

5. Deployment & Polish Guidelines

- **App Icon & Theming:** Define a clear primary color (e.g., Teal or Deep Orange) in your `Color.kt` file. Use Material 3 Dynamic coloring if targeting Android 12+.
- **Error Handling:** The Open Food Facts database does not contain every product on earth. Ensure the UI elegantly handles API 404s by prompting the user to manually enter the item name and take a local photo if the product is not found.
- **Exporting Data:** Consider adding a feature to export the Shopping List via Android's native ShareSheet (`Intent.ACTION_SEND`) so users can text the list to a family member.